# RAX-U1
# Gegenprüfung v0.1

Zweiter Auftrag | 14. Oktober 2026 | Interne adversariale Nachprüfung

**Die Mathematik und die gespeicherten statistischen Entscheidungen sind bestätigt. Eine uneingeschränkte technische Abnahme bleibt offen.** Der frühere Archivverlust ist weiterhin ungeklärt; zusätzlich wurde eine Lücke in der Ressourcenprüfung des alten Referenzprüfers reproduziert.

Gegenstand ist die eingefrorene synthetische Kampagne RAX-U1-QUAL-0001. Die Ergebnisse waren vor dieser Gegenprüfung bekannt. Es handelt sich weder um ein externes oder verblindetes Review noch um RAX-U2. Originalcode, Seeds und ursprüngliche Evidenz wurden nicht geändert.

## Was tatsächlich erneut geprüft wurde

| Prüfung | Befund |
|---|---|
| 160 vollständige Übergangsmatrizen | Alle Digests identisch; 32 stark lumpbar, 128 nicht lumpbar. |
| 640 zusätzliche Umbenennungen / 32 verschobene Kontrollen | Invarianten erhalten; verschobene Referenzdynamik erkannt. |
| 3.200.000 Übergänge / 160 Messzellen | Vollständig neu gezählt; alle 6.400.000 Rohbytes im Replay identisch. |
| 40 statistische Entscheidungen | 8 gestützt, 16 verworfen, 16 unentschieden. |
| Numerische Gegenrechnung | 80-stellige Dezimalrechnung; maximale Float-Abweichung $1{,}11 \times 10^{-16}$. |
| Fehler- und Grenzproben | 6 Fehlerproben und 4.128 Grenzentscheidungen; 25 bestehende Tests erneut bestanden. |

## Einordnung

U1 ist eine belastbare Kontrolle einer **absichtlich eingebauten** Markov-Reduktion innerhalb einer festgelegten Generatorfamilie. Die Gegenprüfung stützt diese begrenzte Aussage. Sie liefert keinen Nachweis unbekannter Universalität, emergenter Geometrie oder neuer Physik.

**Abnahmestatus:** Numerische Reproduktion bestätigt; Standalone-Ressourcenprüfung ergänzt; Archivdauerhaftigkeit offen; U2-Entdeckungsgate noch nicht freigegeben.

# 1 | Mathematik und Entscheidungsregeln

## Ein weiterer Rechenweg

Der neue Kern importiert weder u1.py noch reference.py. Er baut die vollständigen Matrizen aus ganzzahligen Zeilenzählern und gemeinsamen Nennern auf. V ordnet Mikrozustände ihren Blöcken zu; U mittelt innerhalb jedes Blocks. Das Kriterium lautet PV = VUPV, mit Q = UPV. Es entspricht dem Kemeny-Snell-Kriterium [1]. Die neue Rechnung bestätigt für alle 160 Fälle die exakten Identitäten: Blockheterogenität Δ = 2λ, maximaler Vorhersagefehler L = λ.

## Zwei Schwellen, aber keine unabhängige Bestätigung

Für die beiden Präparationen eines Ausgangsblocks gilt $|\hat{p}_- - \hat{p}_+| \le |\hat{p}_- - r| + |\hat{p}_+ - r|$. Hier ist $\delta_{Block} = 0{,}05 = 2\delta_{Vorhersage}$. Deshalb impliziert das Bestehen beider Vorhersagegrenzen automatisch das Bestehen der Blockgrenze. Umgekehrt würde eine Verletzung der unteren Blockgrenze mindestens eine verletzte Vorhersagegrenze erzwingen. Die Entscheidungen sind korrekt, aber das zweite Kriterium erhöht hier nicht die Evidenz.

Mit $b = \sqrt{[\log(6400)/40000]} = 0{,}014802071873007984$ vereinfacht sich die gesamte Konfigurationsentscheidung zu: alle vier $|\hat{p} - r| \le 0{,}0101979281$ → gestützt; mindestens eines > 0,0398020719 → verworfen; sonst unentschieden. Dies ist eine Ableitung der bestehenden Regeln, keine nachträgliche Änderung.

## Numerische Binomialkalibrierung

Unter dem idealen Modell unabhängiger Bernoulli-Versuche wurden endliche Binomialsummen berechnet, ohne neue Qualifikationsdaten zu ziehen. SciPy-CDF/SF [3] und eine unabhängige Rekurrenz vom Verteilungsmodus stimmen bis $4{,}22 \times 10^{-15}$ absolut überein. Wahrscheinlichkeiten gelten je Konfiguration:

| λ | Gestützt | Verworfen | Unentschieden |
|---|---|---|---|
| 0 | 98,7310 % | ca. $2{,}05 \times 10^{-27}$ % | 1,2690 % |
| 0,01 | 7,3256 % | ca. $2{,}36 \times 10^{-15}$ % | 92,6744 % |
| 0,025 | praktisch 0 | 0,003564 % | 99,996436 % |
| 0,05 | praktisch 0 | nahe 100 % | ca. $4{,}77 \times 10^{-10}$ % |
| 0,10 | praktisch 0 | nahe 100 % | praktisch 0 |

Bei λ=0,01 ist die geringe Bestätigungswahrscheinlichkeit eine Präzisionsgrenze des gewählten Designs. Bei λ=0,025 liegt die wahre Reduktion genau auf der zulässigen Grenze; eine Verwerfung ist dort eine Fehlentscheidung. Die numerisch berechnete Vereinigungsgrenze für Intervall-Nichtdeckung aller 160 Originalzellen liegt bei ca. 0,2782 %, unter der vorgegebenen Hoeffding-Garantie von 5 % [2]. Diese Diagnose ersetzt weder das ursprüngliche Fehlerbudget noch die 40 Entscheidungen.

# 2 | Gegenbeispiele und technische Lücken

## Gegenbeispiel: vier passende Mittelwerte reichen nicht

Ein explizites Modell mit acht Zuständen und je vier Zuständen pro Block verwendet die Präparationen {0,1}/{2,3} sowie {4,5}/{6,7}. Die Wechselraten innerhalb der vier Präparationen lauten:

| Präparation | Mikroskopische Wechselraten | Mittelwert |
|---|---|---|
| Block 0, Hälfte − | 0,25 und 0,45 | 0,35 |
| Block 0, Hälfte + | 0,25 und 0,45 | 0,35 |
| Block 1, Hälfte − | 0,35 und 0,55 | 0,45 |
| Block 1, Hälfte + | 0,35 und 0,55 | 0,45 |

Zielzustände werden jeweils gleichverteilt innerhalb des gewählten Zielblocks erreicht. Alle vier Präparationsmittelwerte passen exakt zur Referenz. Ein künstliches Zählfixture mit diesen Mittelwerten besteht die statistische Regel. Dennoch sind $\Delta = 0{,}20$ und $L = 0{,}10$; die Projektion ist nicht stark lumpbar. Der exakte Prüfer erkennt das. Die vollständige rationale Matrix steht in audit-final.json.

**Konsequenz:** Das Beispiel liegt ausserhalb der eingefrorenen U1-Familie und widerlegt deren Resultate nicht. Es widerlegt die Verallgemeinerung von vier Präparationsmittelwerten auf beliebige mikroskopische Maximalfehler. U2 benötigt gezielte heterogene Präparationen und neue, zurückgehaltene Mikrorealisationen.

## Reproduzierte Lücke im bisherigen Referenzprüfer

| Temporär verändertes Metadatum | Hauptprüfer | Alter Referenzprüfer | Neue Prüfung |
|---|---|---|---|
| Laufzeit = 121 s; Grenze 120 s | INVALID | PASS | verworfen |
| Peak-RSS = 262.145 KiB; Grenze 262.144 KiB | INVALID | PASS | verworfen |

Die Originalaufzeichnung meldet 2,468 s und 14.464 KiB und hält die Grenzen ein. Es liegt damit kein nachgewiesener Budgetverstoss der Kampagne vor. Die neue zusätzliche Prüfschicht verwirft die manipulierten Metadaten. Eingefrorene historische Quellen bleiben unverändert; die Korrektur gehört in eine neue Runner-Version.

## Konstante Projektion: das Messereignis bleibt gleich

Eine Ein-Block-Projektion ist immer lumpbar. Sie scheitert hier an der Vorhersage des **ursprünglichen Blockwechsel-Ereignisses**: Der konstante Minimaxfehler bei $e_3/\lambda=0$ ist 0,05. Beim anderen Ziel „nächster ursprünglicher Block ist 1“ wäre er 0,10. Ein Kandidat darf die Vorhersageaufgabe nicht durch sein eigenes konstantes Label ersetzen.

# 3 | Archiv, Reproduktion und nächstes Gate

## Der ursprüngliche Archivvorfall bleibt offen

Die beschädigte Datei enthält 5.680.000 Bytes und ist ein exaktes Präfix der vollständigen Folge von 6.400.000 Bytes. Es fehlen 720.000 Bytes. Das rekonstruierte Archiv passt bytegenau zum bereits zuvor protokollierten Digest. Diese deterministische Wiederherstellung liefert keine neue unabhängige Stichprobe und erklärt nicht den Verlust. Fehlendes fsync im ursprünglichen Schreibpfad ist eine Härtungsstelle, aber kein Nachweis der Ursache.

Ein-Byte-Kürzung, Ein-Byte-Veränderung, manipulierte Zählung und manipulierte gespeicherte Entscheidung werden durch die dafür ausgeführten Prüfungen erkannt. Flush/fsync, erneutes Öffnen, Hashvergleich und Abbruch-/Neustarttests sind vor einer Freigabe für unbeaufsichtigte Erfassung in einer neuen Version zu qualifizieren.

## Reproduzierbares Paket

Das beigefügte Archiv enthält den unveränderten U1-Prüfumfang samt vollständigen und beschädigten Rohdaten, den neuen Gegenprüfer, Fehlerproben, maschinenlesbare Resultate, Quellbindungen und Prüfsummen. Python 3.12.14, NumPy 2.3.5, SciPy 1.17.0 wurden verwendet. Vom Paketwurzelordner:

```
python3 -m pip install -r research/rax_u1_countercheck/requirements.txt
python3 research/rax_u1_countercheck/audit.py --output audit-neu.json
python3 research/rax_u1_countercheck/fault_checks.py faults-neu.json
python3 research/rax_u1/reference.py research/rax_u1/evidence/qualification
python3 -m unittest discover -s research/rax_u1/tests -v
```

Neue Ausgabepfade wählen. Zeitstempel und Laufzeiten dürfen abweichen; Digests, Zählungen und Entscheidungen müssen übereinstimmen. Prüfsummen sichern Dateikonsistenz, nicht Urheberschaft oder externe Präregistrierung.

## Herkunft und Veröffentlichung

Geprüfter lokaler Stand: ee88a0ecfb74b9743eac8edc96c0374d65771437. Die vier eingefrorenen Quelldateien stimmen mit dem Vorab-Implementierungscommit a8b5da3221fbb58e883400bd73923b302fcac55a überein. Das vorhandene Git-Bundle ist prüfbar. Bei Beginn dieser Aufgabe stand der öffentliche Hauptzweig auf 270e224; ein öffentlicher RAX-U1-Zweig war nicht auffindbar. Dieser Bericht behauptet keinen Upload der Gegenprüfung.

## Offene Abnahmebedingungen

**Technik:** Speicherung qualifizieren und Ressourcenchecks in einer neuen Runner-Version vereinheitlichen. **Forschung:** Für U2 feste Messereignisse, zusätzliche heterogene Präparationen, neue Mikrorealisationen, Mehrschritthorizonte und eigenes Fehlerbudget festlegen. Ein internes Replay ersetzt weiterhin keine externe Replikation.

## Methodische Quellen

[1] Tian & Kannan (2006), Lumpability and Commutativity of Markov Processes, Sätze 2.3 und 2.5. Autorenfassung; DOI 10.1080/07362990600632045.
[2] Hoeffding (1963), Probability Inequalities for Sums of Bounded Random Variables. DOI 10.1080/01621459.1963.10500830.
[3] SciPy, scipy.stats.binom, Binomial-PMF, CDF und SF. Abruf: 14.10.2026.